

既然编译选项可以影响程序的行为，在正规比赛中，组织方应提前公布编译选项。如果没有公布，选手最好尽早询问。

A.3.3 gdb 简介

gdb 尽管只是一个文本界面的调试器，但功能十分强大。不管是 **Linux** 和 **Windows** 下的 **MinGW**，**gcc** 和 **gdb** 都是最佳拍档。

gdb 的使用方法很简单——用 **gcc** 编译成 **test.exe** 之后，执行 **gdb test.exe** 即可。不过，如果要用 **gdb** 调试，编译时应加上 **-g** 选项，生成调试用的符号表。

接下来使用 **l** 命令，将看到部分源程序清单。如果用 **l 15**，将会显示第 15 行（以及它前后的若干行）。除此之外，还可以用函数名来定义，如 **l main** 将显示 **main** 函数开头的附近 10 行。如果不加参数执行 **l**，将显示下 10 行；**list -**将显示上 10 行。所有这些操作都可以用 **help list** 命令来查看。**gdb** 中的命令可以简写（例如 **list** 简写成 **l**），大家可以多尝试（提示：试一下命令的前若干个字母）。

运行程序的命令是 **r** (**run**)，但会一直执行到程序结束。如何让它停下来呢？方法是用 **b** (**break**) 命令设置断点。例如，**b main** 命令将在 **main** 函数的开始处设置一个断点，则用 **r** 命令执行时会在这里停下来。如果想继续运行，请用 **c** (**continue**) 命令，而不是继续用 **r** 命令。和 **list** 命令类似，**b** 命令既可以指定行号，也可以在指定函数的首部停下来。笔者在调试很多程序时都是以命令 **b main** 和 **r** 开头的。

如果希望逐条语句地执行程序，不停地用 **b** 和 **c** 命令太麻烦。**gdb** 提供了一些更加方便的指令，其中最常用的有两个：**next**（简写为 **n**）和 **step**（简写为 **s**）。其作用都是执行当前行，区别在于如果当前行涉及函数调用，则 **next** 是把它作为一个整体执行完毕，而 **step** 是进入函数内部。尽管 **n** 和 **s** 都只有一个字母，但有时还是稍显繁琐。在 **gdb** 中，如果在提示符下直接按 **Enter** 键，等价于再次执行上一条指令，因此如果需要连续执行 **s** 或者 **n**，只需要第一次输入该命令，然后直接连接 **Enter** 键即可。另外，和命令行一样，可以按上下箭头来使用历史记录。

另一个常用命令是 **until**（简写为 **u**），让程序执行到指定位置。例如，**u 9** 就是执行到第 9 行，**u doit** 就是执行到 **doit** 函数的开头位置。

停下来以后便打印一些函数值，看看是否和想象的一致。用 **p** (**print**) 命令可以打印出一些变量的值，而 **info locals**（可以简写为 **i lo**）可以显示所有局部变量。如果希望每次程序停下来，则可以用 **display**（简写为 **disp**）命令。例如，**display i+1** 就可以方便地读取 **i+1** 的值。它往往和 **n**、**s** 和 **u** 等单步执行指令配合使用。如果需要列出所有 **display**，可以用 **info display**（简写为 **i disp**）；还可以删除或者临时禁止/恢复一些 **display**，相应的命令为 **delete display** (**d disp**)、**disable display** (**dis disp**) 和 **enable display** (**en disp**)。类似地，也可以根据断点编号删除、禁止和恢复断点，还可以用 **clear** (**cl**) 命令，像 **b** 命令一样根据行号或者函数名直接删除断点。

在多数情况下，灵活运用上述功能已经能高效地调试程序了。下面把涉及的命令列出，供读者参考，如附表 A-2 所示。